04. 크로스체인 기능 분석

개요

Composable Finance는 다양한 블록체인 네트워크 간의 상호운용성을 제공하는 크로스체인 인프라를 구축합니다. 이 문서는 XCM(Cross-Consensus Message Format), IBC(Inter-Blockchain Communication), 그리고 멀티홉 기능을 통한 크로스체인 자산 전송과 메시지 라우팅 메커니즘을 상세히 분석합니다.

4.1 XCM 구현 분석

4.1.1 XCM 설정 및 구성

Composable 런타임에서 XCM은 `xcmp.rs` 파일에서 구성되며, 다음과 같은 핵심 컴포넌트들로 구성됩니다:

```
// XCM 실행기 설정

pub struct XcmConfig;

impl xcm_executor::Config for XcmConfig {

type RuntimeCall = RuntimeCall;

type XcmSender = XcmRouter;

type AssetTransactor = LocalAssetTransactor;

type OriginConverter = XcmOriginToTransactDispatchOrigin;

type IsReserve = MultiNativeAsset<AbsoluteReserveProvider>;

type IsTeleporter = ();
```

```
type UniversalLocation = UniversalLocation;

type Barrier = Barrier;

type Weigher = FixedWeightBounds<UnitWeightCost, RuntimeCall, MaxInstructions>;

type Trader = Trader;

type ResponseHandler = XcmExecutorHandler;

// ... 기타 설정들

}
```

4.1.2 XCM 라우터 및 메시지 전송

XCM 메시지는 두 가지 주요 경로를 통해 라우팅됩니다:

```
/// XCM 메시지 라우팅을 위한 라우터

pub  type XcmRouter = (

// UMP를 사용하여 릴레이체인과 통신

cumulus_primitives_utility::ParentAsUmp<ParachainSystem, PolkadotXcm, ()>,

// XCMP를 사용하여 형제 파라체인과 통신

XcmpQueue,

);
```

UMP (Upward Message Passing): 파라체인에서 릴레이체인으로 메시지 전송

XCMP (Cross-Chain Message Passing): 파라체인 간 직접 메시지 전송

4.1.3 MultiLocation 처리 및 계정 변환

XCM에서 `MultiLocation` 은 크로스체인 주소를 나타내는 핵심 타입입니다:

```
/// MultiLocation을 AccountId로 변환하는 타입
pub type LocationToAccountId = (
// 부모(릴레이체인) 오리진을 부모 AccountId로 변환
ParentIsPreset<AccountId>,
// 형제 파라체인 오리진을 ParaId를 통해 AccountId로 변환
SiblingParachainConvertsVia<Sibling, AccountId>,
// 직접적인 AccountId32 오리진을 AccountId로 별칭 처리
AccountId32Aliases<RelayNetwork, AccountId>,
// 멀티홉 트랜잭션을 위한 AccountId32 변환
AccountId32MultihopTx<AccountId>,
);
```

멀티홉 트랜잭션을 위한 특별한 변환 로직:

```
pub struct AccountId32MultihopTx<AccountId>(PhantomData<AccountId>);
impl<AccountId: From<[u8; 32]> + Into<[u8; 32]> + Clone>
xcm_executor::traits::Convert<MultiLocation, AccountId> for AccountId32MultihopT
{
fn convert(location: MultiLocation) -> Result<AccountId, MultiLocation> {
let id = match location {
MultiLocation {
parents: 0,
interior: X4(
```

```
PalletInstance(_),
GeneralIndex(_),
AccountId32 { id, network: _ },
AccountId32 { id: _, network: _ },
),
} => id,
// X5, X6, X7, X8 패턴들도 지원
_ => return Err(location),
};
Ok(id.into())
}
}
```

4.1.4 자산 거래 및 수수료 처리

XCM 자산 거래는 `LocalAssetTransactor` 를 통해 처리됩니다:

```
pub  type LocalAssetTransactor = MultiCurrencyAdapterWrapper<
crate::Assets,
UnknownTokens,
IsNativeConcrete<CurrencyId, AssetsIdConverter>,
AccountId,
LocationToAccountId,
AssetsIdConverter,
```

```
DepositToAlternative<TreasuryAccount, Tokens, CurrencyId, AccountId, Balance>,
PalletMultihopXcmIbc,
>;
```

수수료 처리를 위한 트레이더 설정:

```
pub type Trader = TransactionFeePoolTrader<
AssetsIdConverter,
FinalPriceConverter,
ToTreasury<AssetsIdConverter, crate::Assets, TreasuryAccount>,
WeightToFeeConverter,
>;
```

4.1.5 XCM 배리어 및 보안

XCM 실행을 위한 보안 배리어:

```
pub type Barrier = (
TakeWeightCredit,
AllowKnownQueryResponsesSubstituteQuerierIfNone<PolkadotXcm>,
AllowSubscriptionsFrom<ParentOrSiblings>,
AllowTopLevelPaidExecutionFrom<Everything>,
);
```

4.2 IBC 프로토콜 구현 분석

4.2.1 IBC 설정 및 구성

IBC 프로토콜은 `ibc.rs` 파일에서 구성되며, Cosmos 생태계와의 상호운용성을 제공합니다:

```
impl pallet_ibc::Config for Runtime {
type TimeProvider = Timestamp;
type RuntimeEvent = RuntimeEvent;
type NativeCurrency = Balances;
type Balance = Balance;
type AssetId = CurrencyId;
type NativeAssetId = NativeAssetId;
type IbcDenomToAssetIdConversion = IbcDenomToAssetIdConversion;
type PalletPrefix = IbcTriePrefix;
type LightClientProtocol = GRANDPA;
type AccountIdConversion = ibc_primitives::IbcAccount<AccountId>;
type Fungibles = Assets;
type Router = Router;
type MinimumConnectionDelay = ConstU64<1>;
type ParaId = parachain_info::Pallet<Runtime>;
type RelayChain = RelayChainId;
// ... 기타 설정들
}
```

4.2.2 IBC 라우터 및 모듈 관리

IBC 라우터는 다양한 IBC 모듈을 관리합니다:

```
#[derive(Clone, Debug, Eq, PartialEq, Default)]
pub struct Router {
ics20: pallet_ibc::ics20::memo::Memo<
Runtime,
pallet_ibc::ics20_fee::Ics20ServiceCharge<Runtime, pallet_ibc::ics20::IbcModule<
>,
}

impl ModuleRouter for Router {
fn get_route_mut(&mut self, module_id: &ModuleId) -> Option<&mut dyn Module>
match module_id.as_ref() {
MODULE_ID_STR => Some(&mut self.ics20),
&_ => None,
}
}

fn has_route(module_id: &ModuleId) -> bool {
matches!(module_id.as_ref(), MODULE_ID_STR)
}
```

```
fn lookup_module_by_port(port_id: &PortId) -> Option<ModuleId> {
match port_id.as_str() {
PORT_ID_STR => ModuleId::from_str(MODULE_ID_STR).ok(),
_ => None,
}
}
}
```

4.2.3 IBC 자산 변환 및 관리

IBC 디놈(denomination)과 자산 ID 간의 변환:

```
pub struct IbcDenomToAssetIdConversion;

impl DenomToAssetId<Runtime> for IbcDenomToAssetIdConversion {
type Error = DispatchError;

fn from_denom_to_asset_id(denom: &str) -> Result<CurrencyId, Self::Error> {
ForeignIbcIcs20Assets::<AssetsRegistry>::from_denom_to_asset_id(denom)
}

fn from_asset_id_to_denom(id: CurrencyId) -> Option<String> {
ForeignIbcIcs20Assets::<AssetsRegistry>::from_asset_id_to_denom(id)
}
```

```
fn ibc_assets(start_key: Option<Either<CurrencyId, u32>>, limit: u64) -> IbcAss
// IBC 자산 목록 반환 로직
let mut iterator = match start_key {
None => IbcAssetIds::<Runtime>::iter().skip(0),
Some(Either::Left(asset_id)) => {
let raw_key = asset_id.encode();
IbcAssetIds::<Runtime>::iter_from(raw_key).skip(0)
},
Some(Either::Right(offset)) => IbcAssetIds::<Runtime>::iter().skip(offset as us
};
// ... 자산 목록 구성 로직
}
}
```

4.2.4 라이트 클라이언트 및 상태 검증

IBC는 GRANDPA 라이트 클라이언트를 사용하여 상태를 검증합니다:

```
parameter_types! {
pub const GRANDPA: pallet_ibc::LightClientProtocol = pallet_ibc::LightClientPro
pub const RelayChainId: RelayChain = RelayChain::Rococo;
}
```

4.3 멀티홉 XCM-IBC 기능 분석

4.3.1 멀티홉 라우팅 구조

`pallet_multihop_xcm_ibc` 는 XCM과 IBC를 결합한 크로스체인 전송을 제공합니다:

```
#[pallet::storage]
#[pallet::getter(fn route_id_to_miltihop_path)]
pub type RouteIdToRoutePath<T: Config> = StorageMap<
_,
Blake2_128Concat,
u128, /* route id */
RouteBoundedVec<T>, /* route to forward */
OptionQuery,
>;
```

라우트 추가 함수:

```
#[pallet::call_index(0)]
#[pallet::weight(100_000)]
pub fn add_route(
origin: OriginFor<T>,
route_id: u128,
route: RouteBoundedVec<T>,
) -> DispatchResult {
```

```
ensure_root(origin)?;

RouteIdToRoutePath::<T>::insert(route_id, route);

Ok(())

}
```

4.3.2 XCM에서 IBC로의 전환

멀티홉 XCM 핸들러는 XCM 전송을 처리합니다:

```
impl<T: Config> pallet_ibc::ics20::SubstrateMultihopXcmHandler for Pallet<T>

where

u128: Into<<T as  orml_xtokens::Config>::Balance>,

u128: Into<<T as  orml_xtokens::Config>::CurrencyId>,

{

type AccountId = T::AccountId;



fn  transfer_xcm(

from: T::AccountId,

to: T::AccountId,

para_id: Option<u32>,

amount: u128,

currency: u128,

) -> Option<()> {

let  signed_account_id = RawOrigin::Signed(from.clone());
```

```
let  acc_bytes = T::AccountId::encode(&to);

let Ok(id) = acc_bytes.try_into() else {

// 에러 이벤트 발생

return None

};



// MultiLocation 구성

let  location = if  let Some(parachain_id) = para_id {

xcm::latest::MultiLocation::new(

1, // parent

xcm::latest::Junctions::X2(

xcm::latest::Junction::Parachain(parachain_id),

xcm::latest::Junction::AccountId32 { id, network: None },

),

)

} else {

xcm::latest::MultiLocation::new(

1, // parent

xcm::latest::Junctions::X1(xcm::latest::Junction::AccountId32 {

id,

network: None,

}),

)
```

```
};

// XCM 전송 실행
let result = orml_xtokens::Pallet::<T>::transfer(
signed_account_id.into(),
currency.into(),
amount.into(),
Box::new(location.into()),
WeightLimit::Unlimited,
);

result.ok()
}
}
```

4.3.3 멀티홉 메모 생성 및 처리

복잡한 멀티홉 경로를 위한 메모 생성:

```
pub fn create_memo(
mut list_chain_name_address: ListChainNameAddress,
) -> Result<Option<Memo>, DispatchError> {
list_chain_name_address.reverse(); // 역순으로 메모 생성
```

```
let mut last_memo_data: Option<Memo> = None;


for (i, name, address) in list_chain_name_address {
let mut forward = if i.chain_hop == ChainHop::Xcm {
let memo_receiver = scale_info::prelude::format!("0x{}", hex::encode(address));
ForwardingMemo::new_xcm_memo(memo_receiver, XcmHop::new(i.para_id))
} else {
let memo_receiver = if i.chain_hop == ChainHop::SubstrateIbc {
scale_info::prelude::format!("0x{}", hex::encode(address))
} else {
// Cosmos 주소를 위한 bech32 인코딩
let data: Vec<u5> = address
.into_iter()
.map(|byte| u5::try_from_u8(byte))
.collect::<Result<_, _>>()?;


let name = String::from_utf8(name.clone())?;
bech32_no_std::encode(&name, data, bech32_no_std::Variant::Bech32)?
};


ForwardingMemo::new_ibc_memo(
memo_receiver,
PortId::transfer(),
```

```
ChannelId::new(i.channel_id),
i.timeout.ok_or(DispatchError::Other("Timeout is none"))?,
i.retries.ok_or(DispatchError::Other("Retries is none"))?,
)
};

if let Some(memo_memo) = last_memo_data {
forward.next = Some(Box::new(memo_memo));
};
let new_memo = Memo::forward(forward);
last_memo_data = Some(new_memo);
}
Ok(last_memo_data)
}
```

4.3.4 멀티홉 자산 예치 콜백

XCM 자산 예치 후 IBC 전송을 트리거하는 콜백:

```
impl<T: Config> MultiCurrencyCallback for Pallet<T> {
type AssetId = T::AssetId;

fn deposit_asset(
asset: &xcm::latest::MultiAsset,
```

```
location: &xcm::latest::MultiLocation,

_context: &xcm::latest::XcmContext,

deposit_result: xcm::latest::Result,

asset_id: Self::AssetId,

) -> Option<()> {

deposit_result.ok()?;

// MultiLocation 파싱 (X4, X5, X6, X7, X8 지원)

let  location_info = match  location {

MultiLocation {

parents: 0,

interior: X4(

PalletInstance(pallet_id),

GeneralIndex(route_id),

AccountId32 { id: current_network_address, network: _ },

AccountId32 { id: ibc1, network: _ },

),

} => (pallet_id, *current_network_address, *route_id, sp_std::vec![*ibc1]),

// ... 다른 패턴들

_ => {

<Pallet<T>>::deposit_event(crate::Event::<T>::FailedMatchLocation {});

return None

},

};
```

```
// 라우트 검증 및 IBC 전송 실행

// ... 라우트 처리 로직

Some(())

}

}
```

4.4 크로스체인 전송 플로우

4.4.1 XCM 전송 플로우

1. **메시지 구성**: XCM 메시지가 구성되고 필요한 자산과 수수료가 계산됩니다
2. **배리어 검사**: 보안 배리어를 통과하여 실행 권한을 확인합니다
3. **자산 인출**: `WithdrawAsset` 명령으로 자산을 인출합니다
4. **실행 구매**: `BuyExecution` 으로 실행 권한을 구매합니다
5. **자산 예치**: `DepositAsset` 으로 대상 계정에 자산을 예치합니다

4.4.2 IBC 전송 플로우

1. **패킷 생성**: ICS-20 토큰 전송 패킷이 생성됩니다
2. **채널 검증**: 연결된 채널과 포트를 통해 패킷이 전송됩니다
3. **라이트 클라이언트 검증**: 대상 체인의 상태가 검증됩니다
4. **패킷 처리**: 대상 체인에서 패킷을 수신하고 처리합니다
5. **확인 응답**: 성공/실패 확인 응답이 원본 체인으로 전송됩니다

4.4.3 멀티홉 전송 플로우

1. **라우트 설정**: 관리자가 멀티홉 라우트를 설정합니다
2. **XCM 시작**: 사용자가 XCM 전송을 시작합니다
3. **콜백 트리거**: XCM 예치 후 멀티홉 콜백이 트리거됩니다
4. **메모 생성**: 복잡한 라우팅 메모가 생성됩니다
5. **IBC 전송**: 메모와 함께 IBC 전송이 실행됩니다
6. **체인 간 홉**: 각 중간 체인에서 메모에 따라 다음 홉으로 전송됩니다

4.5 보안 및 에러 처리

4.5.1 XCM 보안 메커니즘

- **배리어 시스템**: 악의적인 XCM 실행을 방지
- **수수료 검증**: 충분한 수수료 지불 확인
- **오리진 검증**: 메시지 발신자의 권한 확인

4.5.2 IBC 보안 메커니즘

- **라이트 클라이언트 검증**: 상대방 체인의 상태 검증
- **타임아웃 처리**: 패킷 전송 시간 제한
- **스팸 방지**: 예치금을 통한 스팸 방지

4.5.3 에러 처리 및 복구

- **실패 이벤트**: 모든 실패 상황에 대한 이벤트 발생
- **자산 복구**: 실패한 전송의 자산 복구 메커니즘
- **재시도 로직**: 일시적 실패에 대한 재시도 메커니즘

이 크로스체인 기능 분석을 통해 Composable Finance가 어떻게 다양한 블록체인 네트워크 간의 상호운용성을 달성하는지 이해할 수 있습니다. XCM과 IBC의 결합을 통해 Polkadot/Kusama 생태계와

Cosmos 생태계를 연결하는 강력한 인프라를 제공합니다.